

L&T Edutech - MSc Artificial Intelligence

Task 10

## **Containerizing Flask Deep Learning API**

Deployment Report

|                     |                                                                                                   |
|---------------------|---------------------------------------------------------------------------------------------------|
| Task                | 10 - Containerizing Flask Deep Learning API                                                       |
| Objective           | Package a Flask-based deep learning API into a Docker container for local deployment.             |
| Implementation      | Flask REST API + TensorFlow/Keras CNN + Docker                                                    |
| Submission Evidence | Source code, Dockerfile, build output, container runtime, API tests, screenshots and observations |
| Tested Port         | 5000:5000                                                                                         |

# Table of Contents

| Section | Title                                  | Page  |
|---------|----------------------------------------|-------|
| 1       | Introduction and Objectives            | 3     |
| 2       | Tools, Components and Architecture     | 4     |
| 3       | Implementation and Dockerfile          | 5     |
| 4       | 1 - Docker Image Build                 | 6     |
| 5       | 2 - Container Running and Prediction   | 7     |
| 6       | 3 - Health Check                       | 8     |
| 7       | 4 - Resource Usage                     | 9     |
| 8       | 5 - Image Size                         | 10    |
| 9       | 6 - Docker History                     | 11    |
| 10      | 7 - Automated API Validation           | 12    |
| 11      | Results, Observations and Optimization | 13    |
| 12      | Source Code Appendix                   | 14-16 |
| 13      | Conclusion                             | 17    |

## Evidence sequence used in this report

|   |                                                                                    |
|---|------------------------------------------------------------------------------------|
| 1 | Docker build completed and image tagged task10-flask-dl-api: latest                |
| 2 | Docker container running with port mapping 5000:5000 and prediction response for 7 |
| 3 | Health endpoint returned success=true, status=healthy and model loaded=true        |
| 4 | Container resource usage captured with docker stats                                |
| 5 | Docker image disk/content size captured                                            |
| 6 | Docker image layers inspected using docker history                                 |
| 7 | Automated API smoke test returned HTTP 200 for /health and /                       |

# 1. Introduction and Objectives

This practical task packages the Flask-based MNIST deep learning inference API developed in the previous Flask/Streamlit task into a Docker container. The container provides an isolated runtime for the Flask service, the TensorFlow/Keras model and its Python dependencies, while exposing the API through host port 5000.

The previous implementation exposes three endpoints: GET / for service information, GET /health for readiness, and POST /predict for image inference. The backend converts an uploaded image to grayscale, fits it to 28 x 28 pixels, normalizes pixel values to the range [0, 1], invokes the saved CNN model and returns prediction data as JSON. This report uses the project source and the observed screenshots as its evidence basis.

## Objectives

- Create a Dockerfile that packages the Flask API and compatible trained model.
- Build a Docker image named task10-flask-dl-api.
- Run the image as a container with host-to-container port mapping 5000:5000.
- Verify the root, health and prediction API functionality.
- Inspect resource usage and image layers and document optimization practices.
- Produce a deployment report containing source code, screenshots, outputs, results and observations.

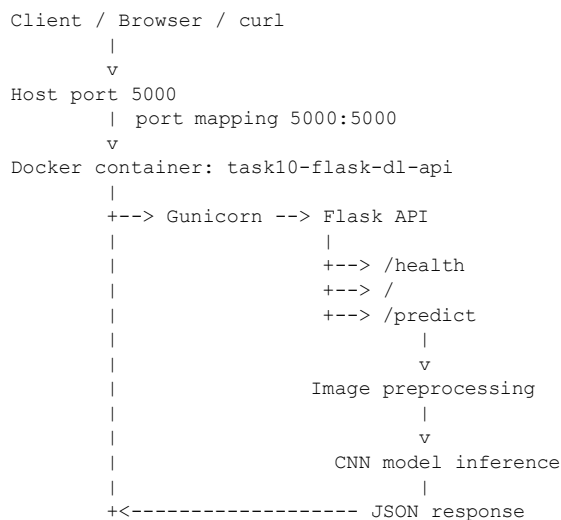
## 2. Tools, Components and Architecture

The implementation uses the Flask inference service and the same model-driven workflow established in the previous task. The Docker container is the deployment boundary; the host browser or curl client acts as the API consumer.

### Technology stack

| Component                     | Role                                                 |
|-------------------------------|------------------------------------------------------|
| Python 3.12                   | Application runtime used in the Docker image         |
| Flask 3.1.2                   | REST API framework                                   |
| TensorFlow CPU 2.21.0 / Keras | Loads the CNN model and performs inference           |
| Pillow + NumPy                | Image decoding, resizing and numerical preprocessing |
| Gunicorn 26.2.0               | WSGI server used as the container entrypoint         |
| Docker Desktop                | Builds and runs the Linux container on Windows       |

### Architecture flow



The deployment therefore keeps the model and API inside the container while the Docker port mapping makes the service available locally at <http://127.0.0.1:5000>.

### 3. Implementation and Dockerfile

The Dockerfile uses a two-stage build. A builder stage creates a Python virtual environment and installs the pinned dependencies without retaining pip cache data. The runtime stage starts from python:3.12-slim, installs only the required libgomp1 runtime library, copies the prepared virtual environment, application code and model, and runs the service as a non-root user.

#### Key deployment features

- Slim Python base image.
- Two-stage build to separate dependency preparation from runtime contents.
- `pip --no-cache-dir` to avoid package-cache retention.
- Non-root user `appuser`.
- EXPOSE 5000 and Docker HEALTHCHECK for service readiness.
- Gunicorn rather than the Flask development server for the container runtime.

#### Dockerfile used for the tested image

```
# syntax=docker/dockerfile:1

FROM python:3.12-slim AS builder
ENV PYTHONDONTWRITEBYTECODE=1 PYTHONUNBUFFERED=1 PIP_DISABLE_PIP_VERSION_CHECK=1
WORKDIR /build
RUN python -m venv /opt/venv
ENV PATH="/opt/venv/bin:$PATH"
COPY requirements.txt .
RUN pip install --upgrade pip && pip install --no-cache-dir -r requirements.txt

FROM python:3.12-slim AS runtime
ENV PYTHONDONTWRITEBYTECODE=1 PYTHONUNBUFFERED=1 PIP_DISABLE_PIP_VERSION_CHECK=1 PATH="/opt/venv/bin:$PATH"
WORKDIR /app
RUN apt-get update && apt-get install -y --no-install-recommends libgomp1 && rm -rf /var/lib/apt/lists/*
COPY --from=builder /opt/venv /opt/venv
COPY flask_api.py .
COPY deep_learning_model.h5 .
RUN useradd --create-home --uid 10001 appuser && chown -R appuser:appuser /app
USER appuser
EXPOSE 5000
HEALTHCHECK --interval=30s --timeout=5s --start-period=30s --retries=3 CMD python -c "import urllib.request;
CMD ["gunicorn", "--bind", "0.0.0.0:5000", "--workers", "1", "--threads", "2", "--timeout", "120", "flask_api:app"]
```

## 4. Evidence 1 - Docker Image Build

The build command used was `docker build --pull -t task10-flask-dl-api ..`. The supplied screenshot shows a completed BuildKit export and the image tagged as `task10-flask-dl-api:latest`.

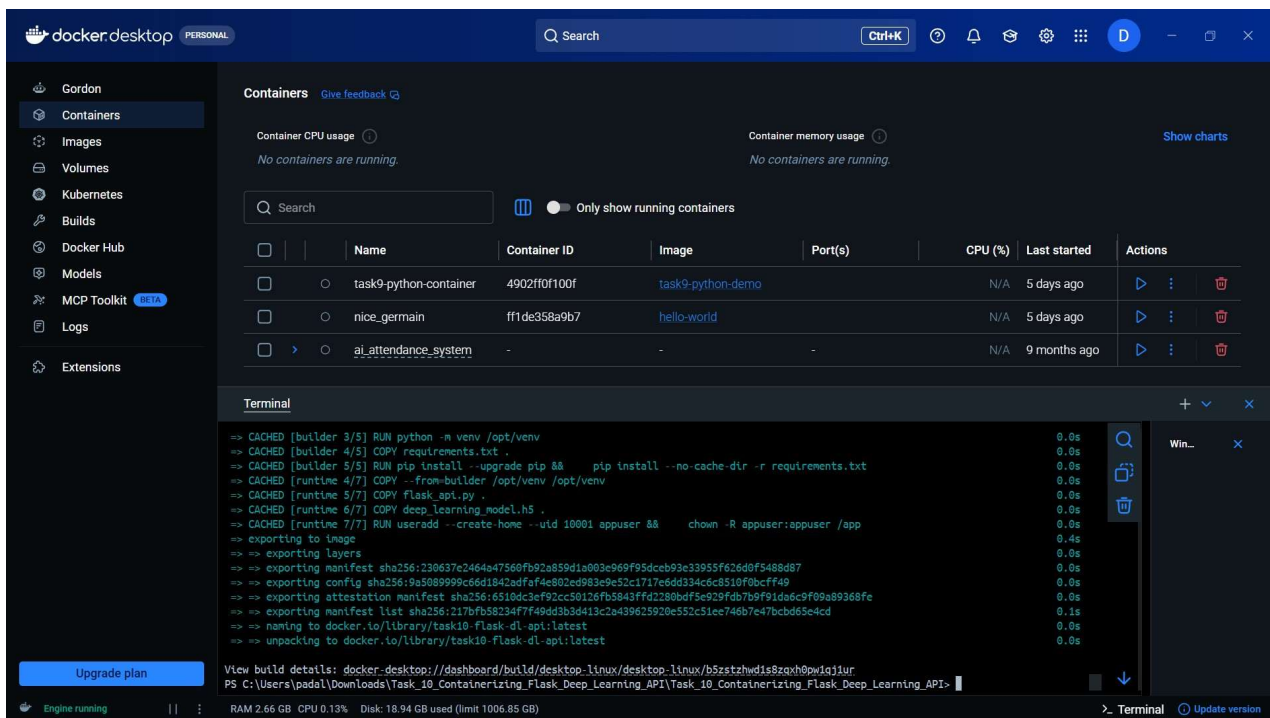
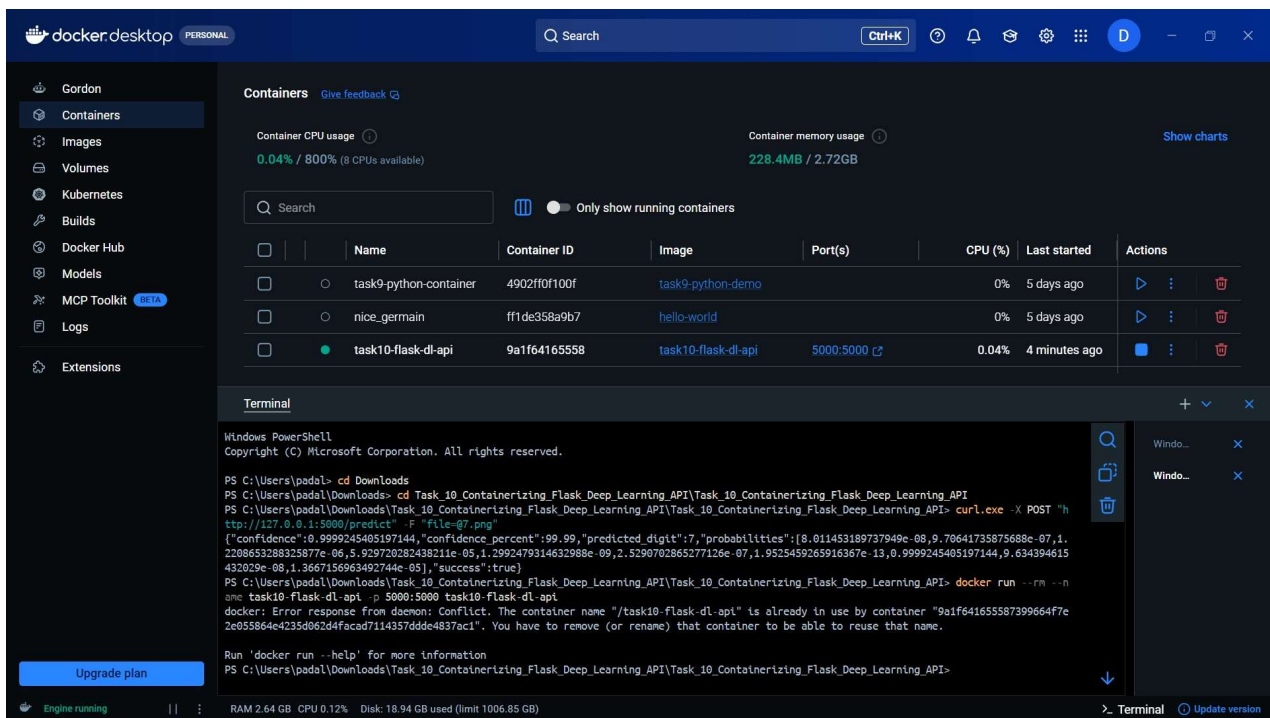


Figure 1. Docker image build completed successfully; the output shows application/model layers and the final tag `task10-flask-dl-api:latest`.

## 5. Evidence 2 - Container Running and Prediction

The container was launched with `docker run --rm --name task10-flask-dl-api -p 5000:5000 task10-flask-dl-api`. The same evidence screen also shows the POST `/predict` call succeeding for 7.png.



The screenshot shows the Docker Desktop interface. On the left is a sidebar with navigation options: Gordon, Containers, Images, Volumes, Kubernetes, Builds, Docker Hub, Models, MCP Toolkit (beta), Logs, and Extensions. The main area is titled 'Containers' and shows a table of running containers. The 'task10-flask-dl-api' container is highlighted, showing it is running with image 'task10-flask-dl-api' and port mapping '5000:5000'. Below the containers list is a terminal window showing a Windows PowerShell session. The terminal output includes a successful POST request to the /predict endpoint for '7.png', returning a confidence of 99.99 and a predicted digit of 7. A subsequent 'docker run' command fails with an error message: 'Error response from daemon: Conflict. The container name "/task10-flask-dl-api" is already in use by container "9a1f641655587399664f7e2e055864e4235d062d4facad7114357ddde4837ac1". You have to remove (or rename) that container to be able to reuse that name.'

| Name                   | Container ID | Image               | Port(s)   | CPU (%) | Last started  | Actions                |
|------------------------|--------------|---------------------|-----------|---------|---------------|------------------------|
| task9-python-container | 4902ff0f100f | task9-python-demo   |           | 0%      | 5 days ago    | [Play] [Stop] [Delete] |
| nice_germain           | ff1de358a9b7 | hello-world         |           | 0%      | 5 days ago    | [Play] [Stop] [Delete] |
| task10-flask-dl-api    | 9a1f64165558 | task10-flask-dl-api | 5000:5000 | 0.04%   | 4 minutes ago | [Play] [Stop] [Delete] |

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

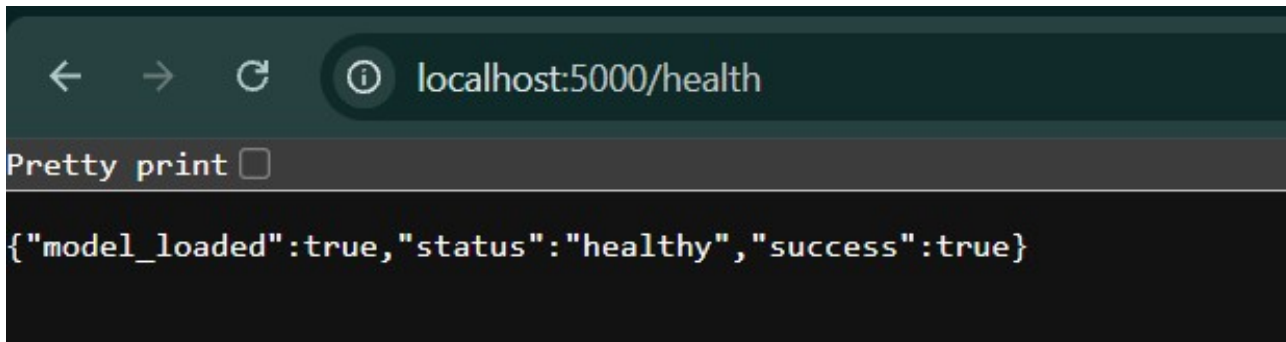
PS C:\Users\paddl> cd Downloads
PS C:\Users\paddl\Downloads> cd Task_10_Containerizing_Flask_Deep_Learning_API\Task_10_Containerizing_Flask_Deep_Learning_API
PS C:\Users\paddl\Downloads\Task_10_Containerizing_Flask_Deep_Learning_API\Task_10_Containerizing_Flask_Deep_Learning_API> curl.exe -X POST "http://127.0.0.1:5000/predict" -F "file=@7.png"
{"confidence":0.9999245405197144,"confidence_percent":99.99,"predicted_digit":7,"probabilities":{"0.011453189737949e-08,9.70641735875688e-07,1.2208653298323877e-06,5.9197720282438211e-05,1.2992479314632988e-09,2.5290702865277126e-07,1.9525459265916367e-13,0.9999245405197144,9.634394615432029e-08,1.366715963492744e-05},"success":true}
PS C:\Users\paddl\Downloads\Task_10_Containerizing_Flask_Deep_Learning_API\Task_10_Containerizing_Flask_Deep_Learning_API> docker run --rm --name task10-flask-dl-api -p 5000:5000 task10-flask-dl-api
docker: Error response from daemon: Conflict. The container name "/task10-flask-dl-api" is already in use by container "9a1f641655587399664f7e2e055864e4235d062d4facad7114357ddde4837ac1". You have to remove (or rename) that container to be able to reuse that name.

Run 'docker run --help' for more information
PS C:\Users\paddl\Downloads\Task_10_Containerizing_Flask_Deep_Learning_API\Task_10_Containerizing_Flask_Deep_Learning_API>
```

Figure 2. Docker Desktop shows task10-flask-dl-api running with port mapping 5000:5000; the terminal first shows a successful /predict request for 7 with success=true and confidence percent=99.99. A later repeated docker run command is also visible and returned a container-name conflict because the tested container was already running.

## 6. Evidence 3 - Health Check

The health endpoint confirms that the Flask process is responding and the model has loaded inside the running container.



*Figure 3. Browser health check at localhost:5000/health returned model loaded=true, status=healthy and success=true.*

## 7. Evidence 4 - Resource Usage

The resource snapshot provides a point-in-time view of CPU, memory, network, block I/O and process count for the ML inference container.

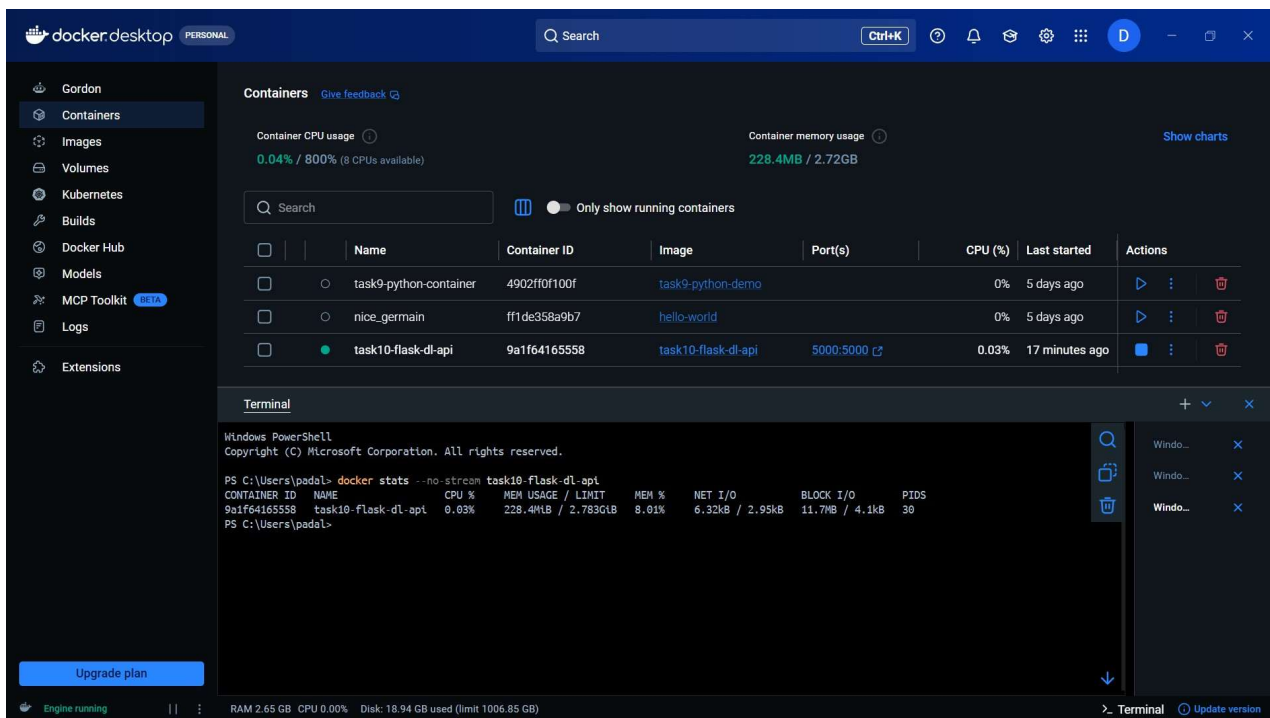


Figure 4. *docker stats* captured 0.03% CPU usage and 228.4 MiB memory usage (8.01% of the displayed limit), with network/block I/O and PID information.

## 8. Evidence 5 - Image Size

The measured size is reported exactly as displayed. The disk-usage footprint is substantially larger than the raw content-size figure, which is expected when image layers and runtime filesystem accounting are included.

The screenshot shows the Docker Desktop interface. The left sidebar contains navigation options: Gordon, Containers, Images, Volumes, Kubernetes, Builds, Docker Hub, Models, MCP Toolkit (beta), Logs, and Extensions. The main area displays 'Containers' with a search bar and a toggle for 'Only show running containers'. Below this is a table of containers:

| Name                   | Container ID | Image               | Port(s)   | CPU (%) | Last started   | Actions                |
|------------------------|--------------|---------------------|-----------|---------|----------------|------------------------|
| task9-python-container | 4902ff0f100f | task9-python-demo   |           | 0%      | 5 days ago     | [Play] [Info] [Delete] |
| nice_germain           | ff1de358a9b7 | hello-world         |           | 0%      | 5 days ago     | [Play] [Info] [Delete] |
| task10-flask-dl-api    | 9a1f64165558 | task10-flask-dl-api | 5000:5000 | 0.03%   | 19 minutes ago | [Play] [Info] [Delete] |
| ai_attendance_system   | -            | -                   | -         | 0%      | 9 months ago   | [Play] [Info] [Delete] |

Below the table, a terminal window is open showing the command `docker images task10-flask-dl-api` and its output:

```
PS C:\Users\padoL> docker images task10-flask-dl-api

IMAGE                ID                DISK USAGE  CONTENT SIZE  EXTRA
task10-flask-dl-api:latest  217fb58234f      2.19GB      415MB         10
PS C:\Users\padoL>
```

The bottom status bar indicates 'Engine running' and shows system metrics: RAM 2.65 GB, CPU 0.37%, and Disk: 18.94 GB used (limit 1006.85 GB).

Figure 5. Docker image size inspection for `task10-flask-dl-api:latest` reported 2.19 GB disk usage and 415 MB content size.

## 9. Evidence 6 - Docker History

The history confirms a structured image with explicit runtime commands and security/readiness configuration. The evidence includes USER appuser, EXPOSE 5000 and HEALTHCHECK.

The screenshot displays the Docker Desktop application window. The left sidebar contains navigation options: Gordon, Containers (selected), Images, Volumes, Kubernetes, Builds, Docker Hub, Models, MCP Toolkit (with a BETA badge), Logs, and Extensions. The main area is titled 'Containers' and shows a summary of container CPU usage (0.03% / 800% (8 CPUs available)) and container memory usage (228.4MB / 2.72GB). Below this is a table of running containers:

| Name                   | Container ID | Image               | Port(s)   | CPU (%) | Last started   | Actions                 |
|------------------------|--------------|---------------------|-----------|---------|----------------|-------------------------|
| task9-python-container | 4902ff0f100f | task9-python-demo   |           | 0%      | 5 days ago     | [Play] [Refresh] [Stop] |
| nice_germain           | ff1de358a9b7 | hello-world         |           | 0%      | 5 days ago     | [Play] [Refresh] [Stop] |
| task10-flask-dl-api    | 9a1f64165558 | task10-flask-dl-api | 5000:5000 | 0.06%   | 21 minutes ago | [Play] [Refresh] [Stop] |

Below the container list is a 'Terminal' section showing the command history for the 'task10-flask-dl-api' container. The terminal output shows the following layers:

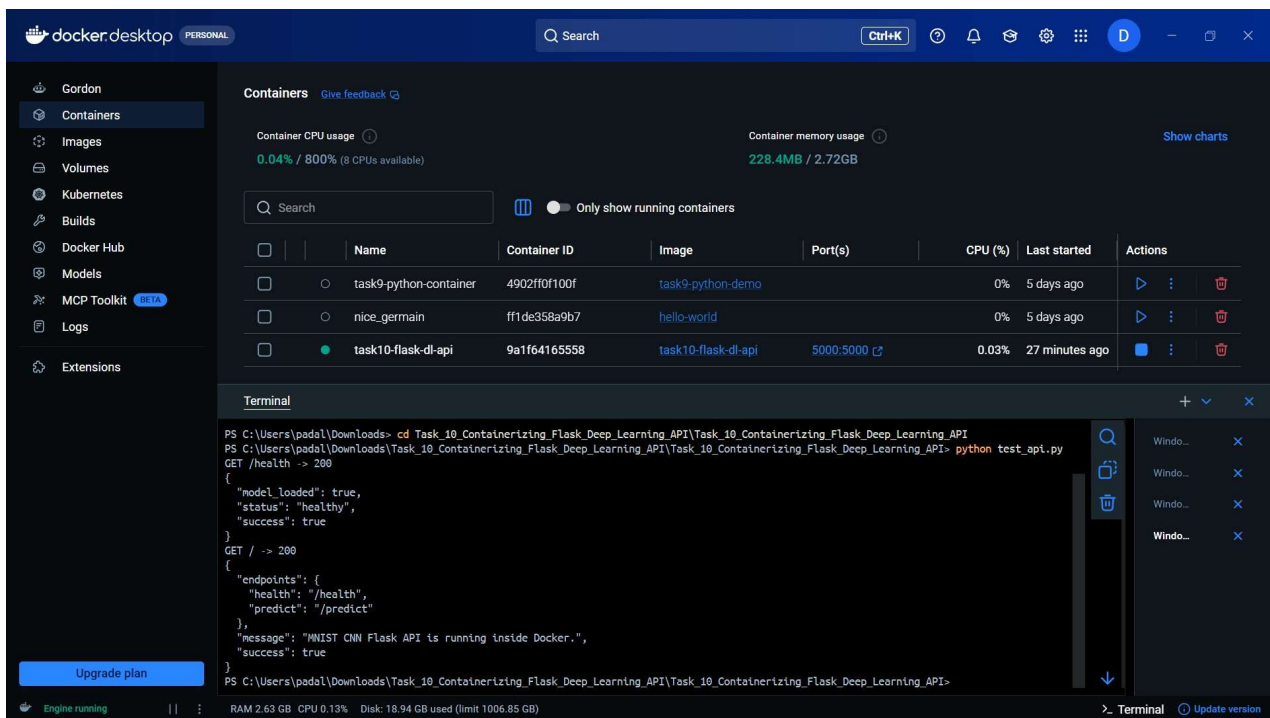
```
PS C:\Users\padal> docker history task10-flask-dl-api
IMAGE          CREATED          CREATED BY          SIZE      COMMENT
217bfb58234f  39 minutes ago  CMD ["python3"]     0B        buildkit.dockerfile.v0
<missing>     39 minutes ago  HEALTHCHECK (Test: 0B        buildkit.dockerfile.v0
EXPOSE [5000/tcp] 0B        buildkit.dockerfile.v0
USER appuser      0B        buildkit.dockerfile.v0
CMD ["python3"]   0B        buildkit.dockerfile.v0
RUN /bin/sh -c set 16.4kB  buildkit.dockerfile.v0
RUN /bin/sh -c set 41.4MB  buildkit.dockerfile.v0
ENV PYTHON_SHA256-c2c4321961fab0fb999d6e0ce0ce... 0B        buildkit.dockerfile.v0
ENV PYTHON_VERSION-3.12.15 0B        buildkit.dockerfile.v0
ENV CPG_KEY-71d9983f63c7513560854A26A821E680... 0B        buildkit.dockerfile.v0
RUN /bin/sh -c set 4.94MB  buildkit.dockerfile.v0
ENV LANG-C.UTF-8 0B        buildkit.dockerfile.v0
ENV PATH=/usr/local/bin:/usr/local/sbin:/usr... 0B        buildkit.dockerfile.v0
# debian.sh --arch 'amd64' out/ 'trixie' '0L... 87.7MB  debuerreotype 0.17
```

The bottom status bar indicates 'Engine running', 'RAM 2.64 GB', 'CPU 10.75%', and 'Disk: 18.94 GB used (limit 1006.85 GB)'. There are also links for 'Terminal' and 'Update version'.

Figure 6. docker history shows the image layers, including Gunicorn CMD, HEALTHCHECK, EXPOSE 5000 and USER appuser.

## 10. Evidence 7 - Automated API Validation

The supplied test\_api.py smoke test exercises the root and health endpoints. Both returned HTTP 200 in the captured run.



The screenshot shows the Docker Desktop interface. The left sidebar contains navigation options: Gordon, Containers (selected), Images, Volumes, Kubernetes, Builds, Docker Hub, Models, MCP Toolkit (BETA), Logs, and Extensions. The main area displays the 'Containers' tab with a search bar and a toggle for 'Only show running containers'. Below this is a table of containers:

|                          | Name                   | Container ID | Image               | Port(s)   | CPU (%) | Last started   | Actions |
|--------------------------|------------------------|--------------|---------------------|-----------|---------|----------------|---------|
| <input type="checkbox"/> | task9-python-container | 4902ff0f100f | task9-python-demo   |           | 0%      | 5 days ago     |         |
| <input type="checkbox"/> | nice_germain           | ff1de358a9b7 | hello-world         |           | 0%      | 5 days ago     |         |
| <input type="checkbox"/> | task10-flask-dl-api    | 9a1f64165558 | task10-flask-dl-api | 5000:5000 | 0.03%   | 27 minutes ago |         |

Below the containers table is a 'Terminal' window showing the execution of a smoke test. The terminal output is as follows:

```
PS C:\Users\podal\Downloads> cd Task_10_Containerizing_Flask_Deep_Learning_API\Task_10_Containerizing_Flask_Deep_Learning_API
PS C:\Users\podal\Downloads\Task_10_Containerizing_Flask_Deep_Learning_API\Task_10_Containerizing_Flask_Deep_Learning_API> python test_api.py
GET /health -> 200
{
  "model loaded": true,
  "status": "healthy",
  "success": true
}
GET / -> 200
{
  "endpoints": {
    "health": "/health",
    "predict": "/predict"
  },
  "message": "MNIST CNN Flask API is running inside Docker.",
  "success": true
}
```

The bottom status bar indicates 'Engine running', 'RAM 2.63 GB', 'CPU 0.13%', and 'Disk: 18.94 GB used (limit 1006.85 GB)'. There is also an 'Upgrade plan' button and a 'Terminal' tab selector.

Figure 7. Automated API smoke test returned GET /health -> 200 with model\_loaded=true and GET / -> 200 with the Dockerized API service message.

## 11. Results, Observations and Optimization

The Dockerized Flask deep learning API was successfully built, started and exercised locally. The captured evidence supports successful image creation, container runtime, health checking, resource inspection and HTTP API validation.

### Observed results

| Test / Item          | Observed result                                           |
|----------------------|-----------------------------------------------------------|
| Docker build         | Image tagged task10-flask-dl-api: latest                  |
| Container runtime    | task10-flask-dl-api running on 5000:5000                  |
| /Predict             | success=true; predicted digit=7; confidence percent=99.99 |
| /Health              | success=true; status=healthy; model loaded=true           |
| docker stats         | 0.03% CPU; 228.4 MiB memory; 8.01% of shown limit         |
| Image size           | 2.19 GB disk usage; 415 MB content size                   |
| Automated smoke test | GET /health -> 200; GET / -> 200                          |

### Optimization practices used

- python:3.12-slim reduces base-image footprint compared with a full Python distribution.
- A multi-stage build keeps builder-only steps separate from the runtime stage.
- pip --no-cache-dir prevents retaining package caches in the built layers.
- .docker ignore limits the build context by excluding local caches, reports, screenshots and unrelated files.
- The runtime runs as a non-root user (appuser).
- Gunicorn is the runtime WSGI server, and one worker is used by default because additional ML workers can increase model memory consumption.
- HEALTHCHECK provides a container-level readiness probe against /health.

### Observation

Although the image uses several optimization practices, TensorFlow and its native dependencies make ML inference images much larger than lightweight web-only Flask services. The report therefore records the measured 2.19 GB disk-usage figure rather than claiming that the resulting image is small.

## 12. Source Code Appendix

The following source listings correspond to the tested Task 10 project files. The trained model binary is not reproduced inside this PDF; the Docker build evidence shows that `deep_learning_model.h5` was copied during image construction.

### 12.1 requirements.txt

```
Flask==3.1.2
gunicorn==26.2.0
numpy>=1.26,<2.3
Pillow>=11,<13
tensorflow-cpu==2.21.0
```

### 12.2 Docker command sequence

```
docker --version
docker info
dir deep_learning_model.h5
docker build --pull -t task10-flask-dl-api .
docker images task10-flask-dl-api
docker run --rm --name task10-flask-dl-api -p 5000:5000 task10-flask-dl-api
curl.exe http://127.0.0.1:5000/health
curl.exe http://127.0.0.1:5000/
curl.exe -X POST "http://127.0.0.1:5000/predict" -F "file=@7.png"
docker ps
docker stats task10-flask-dl-api --no-stream
docker history task10-flask-dl-api
```

### 12.3 test\_api.py

```
import json
from urllib.request import urlopen
BASE_URL = "http://127.0.0.1:5000"

def get_json(path: str):
    with urlopen(BASE_URL + path, timeout=5) as response:
        return response.status, json.loads(response.read().decode("utf-8"))

if __name__ == "__main__":
    status, payload = get_json("/health")
    print("GET /health ->", status)
    print(json.dumps(payload, indent=2))
    status, payload = get_json("/")
    print("GET / ->", status)
    print(json.dumps(payload, indent=2))
```

## 12. Source Code Appendix - Flask API (Part 1)

```
from pathlib import Path
from io import BytesIO
import os

import numpy as np
from PIL import Image, ImageOps
from flask import Flask, jsonify, request
from tensorflow.keras.models import load_model

BASE_DIR = Path(__file__).resolve().parent
MODEL_PATH = Path(os.getenv("MODEL_PATH", BASE_DIR / "deep_learning_model.h5"))

app = Flask(__name__)
app.config["MAX_CONTENT_LENGTH"] = 10 * 1024 * 1024

if not MODEL_PATH.exists():
    raise FileNotFoundError(
        f"Model file not found: {MODEL_PATH}. "
        "Place deep_learning_model.h5 beside flask_api.py or set MODEL_PATH."
    )

model = load_model(MODEL_PATH, compile=False)
print(f"CNN model loaded successfully from: {MODEL_PATH}")

def preprocess_image(file_bytes: bytes) -> np.ndarray:
    image = Image.open(BytesIO(file_bytes)).convert("L")
    image = ImageOps.fit(image, (28, 28), method=Image.Resampling.LANCZOS)
    pixels = np.asarray(image, dtype=np.float32) / 255.0
    input_shape = model.input_shape
    if len(input_shape) == 4:
        return pixels.reshape(1, 28, 28, 1)
    if len(input_shape) == 3:
        return pixels.reshape(1, 28, 28)
    raise ValueError(f"Unsupported model input shape: {input_shape}")

@app.get("/")
def index():
    return jsonify({
        "success": True,
        "message": "MNIST CNN Flask API is running inside Docker.",
        "endpoints": {"health": "/health", "predict": "/predict"},
    }), 200

@app.get("/health")
def health():
    return jsonify({
        "success": True,
        "status": "healthy",
        "model_loaded": model is not None,
    }), 200
```

## 12. Source Code Appendix - Flask API (Part 2)

```
@app.post("/predict")
def predict():
    if "file" not in request.files:
        return jsonify({
            "success": False,
            "error": "Missing image. Send the uploaded image in form field 'file'.",
        }), 400

    uploaded = request.files["file"]
    if not uploaded.filename:
        return jsonify({"success": False, "error": "No file selected."}), 400

    try:
        image_bytes = uploaded.read()
        if not image_bytes:
            return jsonify({"success": False, "error": "Uploaded file is empty."}), 400

        model_input = preprocess_image(image_bytes)
        probabilities = np.asarray(model.predict(model_input, verbose=0))[0]
        predicted_digit = int(np.argmax(probabilities))
        confidence = float(probabilities[predicted_digit])

        return jsonify({
            "success": True,
            "predicted_digit": predicted_digit,
            "confidence": confidence,
            "confidence_percent": round(confidence * 100, 2),
            "probabilities": [float(p) for p in probabilities],
        }), 200

    except Exception:
        app.logger.exception("Prediction failed")
        return jsonify({
            "success": False,
            "error": "Could not process this image. Please upload a valid PNG, JPG, or JPEG.",
        }), 400

@app.errorhandler(413)
def file_too_large(_error):
    return jsonify({
        "success": False,
        "error": "File is too large (maximum 10 MB).",
    }), 413

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=int(os.getenv("PORT", "5000")), debug=False)
```

The Dockerfile overrides this development fallback by launching Gunicorn with flask\_api:app as its WSGI target.

## 13. Conclusion

Task 10 successfully demonstrates containerization of a Flask deep learning inference API. The tested Docker image was built and tagged correctly, the container was launched with host port 5000 mapped to the Flask service, the API health endpoint confirmed that the model was loaded, the prediction endpoint returned a successful digit-7 result, and automated smoke testing returned HTTP 200 for the root and health endpoints.

The deployment also demonstrates practical optimization and operational practices through a slim Python base image, a multi-stage build, no-cache dependency installation, non-root execution, a Docker health check and a production WSGI server. The measured image size of 2.19 GB disk usage is retained as the actual observed result, reflecting the substantial runtime footprint of the TensorFlow-based inference stack.

### Final validation statement

Based on the screenshot evidence captured during the practical session, the Dockerized Flask deep learning API is operational locally and meets the core evaluation criteria of correct containerization and successful deployment.

### GitHub link for LMS

<https://github.com/Saipadala2003/task10-flask-docker-deep-learning-api>